

Transformers Unpacked: The Architecture Behind Modern LLMs

Transformers are a type of deep learning architecture designed to process sequential data such as text. They were introduced in 2017 in the research paper *Attention Is All You Need*. Before transformers, natural language processing mainly relied on recurrent neural networks such as RNNs and LSTMs. Those models processed text one word at a time, which made training slower and made it difficult to capture relationships between words that were far apart in a sentence. Transformers changed this approach by using a mechanism called **attention**, which allows the model to examine different parts of a sequence and determine which words or tokens are most important to each other.

In simple terms, a transformer converts text into numerical representations, studies the relationships between the tokens in that text, and produces contextual representations that can be used for tasks such as text generation, translation, classification, summarization, question answering, and many others. Most modern Large Language Models, including GPT-style models, BERT, T5, Llama, and many models available through Hugging Face, are based on the transformer architecture.

A major strength of transformers is that they do not treat a word as having exactly the same meaning in every sentence. Instead, they create a contextual representation based on the surrounding words. For example, the word **"bank"** means something different in the sentence *"I deposited money in the bank"* compared with *"We sat on the river bank."* A transformer can use surrounding context to create different internal representations of the same word in these two sentences.

Why Do We Need Transformer Models?

Transformer models were developed to overcome several limitations of earlier sequence-processing neural networks. Before transformers became popular, architectures such as Recurrent Neural Networks, or RNNs, and Long Short-Term Memory networks, or LSTMs, were commonly used for language tasks. These models process a sequence step by step. For example, if a sentence contains ten words, the model generally processes the first word, then the second, then the third, and continues in order. This sequential dependency makes it difficult to perform large amounts of computation in parallel, which limits training speed.

Another limitation of older recurrent models is their difficulty in learning long-range relationships. Consider the sentence *"The book that I bought from the old shop near the railway station was expensive."* To understand that **"was expensive"** refers to **"the book"**, the model must retain information across many intermediate words. RNNs and even LSTMs can struggle when dependencies become very long. Transformers solve this problem more directly because the attention mechanism allows one token to interact with another token even if they are far apart in the sequence.

Transformers are also highly suitable for modern computing hardware such as GPUs and TPUs. During training, many token representations can be processed in parallel rather than strictly one at a time. This makes transformers much more scalable. As researchers increased the amount of training data, model size, and computational resources, transformer-based models continued to improve significantly. This scalability is one of the major reasons transformers became the foundation of modern Large Language Models.

Another reason we need transformers is their flexibility. The same general architecture can support many different natural language processing tasks. Encoder-based transformers can be used for text understanding tasks such as sentiment analysis and classification. Decoder-based transformers can be used for text generation. Encoder-decoder transformers can be used for tasks such as translation

and summarization. This flexibility allows one architectural family to support a very wide range of applications.

Transformers also made **pretraining and transfer learning** much more effective. Instead of training a separate language model from scratch for every task, researchers can train a large transformer model on massive datasets and then adapt it to specific tasks. This approach is central to the Hugging Face ecosystem, where users frequently download pretrained checkpoints and fine-tune or use them directly for downstream applications.

Transfer learning is a machine learning technique where a model developed for one task is reused as the starting point for a model on a second, related task.

How Do Transformers Work?

A transformer processes text through several stages. The first stage is **tokenization**. Computers cannot directly understand text in the same way humans do, so the sentence must first be converted into smaller units called tokens. A token may represent a complete word, part of a word, punctuation, or another text unit depending on the tokenizer being used. For example, the sentence *"Transformers are powerful"* may be divided into tokens and each token is then assigned a numerical token ID from the model's vocabulary.

After tokenization, each token ID is converted into a numerical vector called an **embedding**. An embedding is a dense numerical representation that the neural network can process. Instead of representing a word as a simple integer, the model represents it using a vector containing many learned values. These embeddings allow the model to represent useful properties of tokens mathematically.

The transformer must also know the order of the tokens. Attention alone does not automatically understand whether one token appears before or after another. Therefore, transformers include **positional information**. The original transformer used positional encodings, while many modern models use other techniques such as learned position embeddings or Rotary Positional Embeddings, commonly known as RoPE. Positional information helps the model distinguish between sentences such as *"The dog chased the cat"* and *"The cat chased the dog."*

The next and most important stage is **self-attention**. Self-attention allows each token to examine other relevant tokens in the sequence and determine how strongly they are related. For example, in the sentence *"The cat was hungry because it had not eaten,"* the model must understand that the word **"it"** likely refers to **"cat."** Through attention, the representation of the token "it" can use information from the representation of "cat."

To calculate attention, the transformer creates three vectors for each token: the **Query**, **Key**, and **Value** vectors. These are commonly represented as Q, K, and V. The Query represents what information a token is looking for. The Key represents what information a token can be matched against. The Value contains the information that may be passed to another token. The model compares each query with relevant keys and calculates attention scores. Tokens with stronger relationships receive larger attention weights, so their value vectors contribute more strongly to the resulting representation.

The model first compares Queries and Keys using matrix multiplication. These scores are scaled by the square root of the key dimension so that the values do not become excessively large. A softmax function then converts the scores into normalized attention weights. Finally, these weights are multiplied by the Value vectors to produce a new contextual representation.

Transformers normally use **multi-head attention** instead of performing attention only once. Multi-head attention divides the attention operation into multiple heads. Each attention head learns its own set of

projections and may become useful for recognizing different kinds of relationships. One head might respond strongly to nearby grammatical patterns, while another may capture longer-distance dependencies. The outputs of all attention heads are combined and passed to the next stage of the transformer.

After attention, the token representations pass through a **feed-forward neural network**, also called an MLP or feed-forward layer. This network applies additional mathematical transformations to each token representation. Attention allows information to move between token positions, while the feed-forward network transforms the features within each token's representation.

Transformers also use **residual connections**. A residual connection adds the original input of a sublayer back to its output. This helps information and gradients move through deep networks more effectively. Another important component is **normalization**, such as Layer Normalization or RMSNorm. Normalization helps stabilize the activations in the model and makes large transformer networks easier to train.

These operations are repeated across many transformer layers. A small model may contain only a few layers, while large language models can contain dozens or even hundreds of transformer blocks. As token representations move through these layers, they become increasingly contextual and useful for prediction.

For a language-generation model, the final hidden representations are passed through an output layer that produces **logits** for every token in the model's vocabulary. Logits are raw numerical scores. They can be converted into probabilities, after which a decoding method chooses the next token. The generated token is added to the existing sequence, and the process repeats until the model finishes generating the response.

A simplified way to remember the complete process is:

Text → Tokenization → Token IDs → Embeddings → Positional Information → Self-Attention → Feed-Forward Network → Repeated Transformer Blocks → Output Logits → Token Prediction

Transformer Architecture

The transformer architecture is built from repeated neural-network blocks containing attention mechanisms and feed-forward networks. At a high level, transformer models can be divided into three major architectural families: **encoder-only transformers**, **decoder-only transformers**, and **encoder-decoder transformers**. Each architecture uses the same general transformer principles but handles information differently depending on the task.

The original transformer architecture introduced in *Attention Is All You Need* contained both an **encoder** and a **decoder**. The encoder reads and represents the input sequence, while the decoder uses the encoder's representations to generate an output sequence. This architecture was originally designed for machine translation. For example, the encoder might process an English sentence while the decoder generates the corresponding French sentence.

Encoder Architecture

The encoder is designed to understand and represent an input sequence. It normally uses **bidirectional self-attention**, which means every token can attend to tokens that appear both before and after it. Because the encoder has access to the complete input sequence, it is especially useful for tasks where understanding the entire piece of text is important.

Each encoder layer generally contains a multi-head self-attention mechanism followed by a feed-forward neural network. Residual connections and normalization are applied around these components. Several identical encoder layers are stacked on top of one another so that increasingly sophisticated representations can be learned.

BERT is one of the best-known encoder-only transformer models. It is commonly associated with tasks such as text classification, sentiment analysis, named entity recognition, token classification, and extractive question answering. Encoder models are generally strong when the task requires understanding or representing input rather than generating long sequences of new text.

Decoder Architecture

The decoder is designed for text generation. Unlike a normal encoder, a decoder typically uses **causal or masked self-attention**. This masking prevents the model from seeing future tokens while predicting the next token. For example, when predicting the next token in the sequence *"The capital of France is"*, the model can use all the previous tokens but cannot access the correct future answer during training.

Decoder-only models generate text autoregressively, meaning they generate one token at a time. After predicting one token, that token becomes part of the context used to predict the next token. This process continues until the model reaches a stopping condition.

GPT-style models and Llama are examples of decoder-only transformer architectures. Most general-purpose generative Large Language Models use a decoder-only design because it works naturally for conversational AI, text generation, code generation, completion, and instruction-following tasks.

Encoder-Decoder Architecture

An encoder-decoder transformer combines both components. The encoder processes the entire input sequence and produces contextual representations. The decoder then generates the output sequence while using both its previously generated tokens and the encoder's representations.

A special mechanism called **cross-attention** allows the decoder to attend to information produced by the encoder. In cross-attention, the decoder usually creates the Queries, while the encoder output provides the Keys and Values. This allows the decoder to determine which parts of the input are most relevant while generating each output token.

Models such as T5 and BART use encoder-decoder architectures. These models are especially useful for sequence-to-sequence tasks where one input sequence must be transformed into another output sequence. Common examples include translation, summarization, grammatical correction, and text transformation.

A useful way to remember the three architectures is:

Encoder-only models are mainly designed to understand text. Decoder-only models are mainly designed to generate text. Encoder-decoder models are mainly designed to transform one sequence into another.

Core Components of a Transformer Architecture

The **tokenizer** is responsible for converting raw text into tokens and token IDs. Although technically the tokenizer exists outside the neural-network architecture itself, it is the first major step in using a transformer model.

The **embedding layer** converts token IDs into dense numerical vectors. These vectors form the initial representation of the input tokens.

The **positional mechanism** provides information about token order. Without positional information, the model would have difficulty distinguishing different arrangements of the same tokens.

The **self-attention layer** allows tokens to exchange contextual information. This is the defining mechanism of the transformer architecture.

The **multi-head attention mechanism** performs several attention operations in parallel using different learned projections. This helps the model represent multiple relationships simultaneously.

The **feed-forward neural network** transforms each token representation after attention has mixed information across positions.

The **residual connections** preserve information from previous layers and improve gradient flow during training.

The **normalization layers** help stabilize activations and improve optimization.

The **output layer**, or language modeling head in generative models, converts final hidden states into vocabulary logits that can be used for token prediction.